

Vera AI Challenge — Complete Project Guide

This document explains the **entire project** in plain language: what Magicpin asked for, what we built, every important term, and every step done so far.

Project folder: ~/Projects/vera-ai-challenge

GitHub repo: <https://github.com/lovepreet166/vera-ai-challenge>

Team: Lovepreet / Lovepreet Singh / lovepreetsingh40888@gmail.com

1. What this project is (big picture)

Magicpin

An Indian local-commerce company. People discover nearby shops/restaurants on Magicpin; merchants (shop owners) get customers, offers, and visibility.

Vera

Magicpin's **AI assistant for merchants**. It talks to shop owners on **WhatsApp**, helps them improve Google listings, run campaigns, and sometimes message their customers.

magicpin AI Challenge

A hiring/filter contest. Instead of only a resume/interview, you **build a bot like Vera (or better)** and Magicpin's systems **score it automatically**.

Official page: <https://magicpin.com/vera/ai-challenge>

What you must build

A **public web API** (a bot on the internet) that:

1. Receives business data (category, merchant, triggers, customers)
 2. Decides **when** to message and **what** to say
 3. Handles replies (yes / stop / auto-reply / hostile)
 4. Stays online so Magicpin's **judge** can call it
-

2. Glossary — every important term

Product / business terms

Term	Meaning
Merchant	Shop/clinic/salon owner (e.g. Dr. Meera's Dental Clinic)
Customer	Person who visits that merchant (e.g. patient Priya)
Category / vertical	Type of business: dentists, salons, restaurants, gyms, pharmacies
WhatsApp message	Short chat message Vera would send
CTA (Call To Action)	What you want the person to do next (YES/NO, reply A/B, open question)
Offer	Deal like Dental Cleaning @ ₹299 (service + price works better than vague "10% off")
GBP	Google Business Profile (Google Maps listing)
Hinglish	Mix of Hindi + English, common for Indian merchants
Compulsion / engagement	How likely someone is to reply , not just read
Auto-reply	Phone's canned WhatsApp Business reply ("Thanks for contacting...") — not a real human
Intent	Merchant's real meaning (e.g. "ok let's do it" = start the action)

The 4 context layers (core idea)

Every message is built from up to four JSON "context packs":

Layer	What it is	Example
CategoryContext	Rules for that business type	Dentist tone, taboo words, research digest
MerchantContext	This specific shop's state	CTR 2.1%, offers, owner name Meera
TriggerContext	Why message now	Research digest released, recall due, Diwali
CustomerContext	Optional; for customer-facing msgs	Priya, prefers weekday evenings

Compose = turn those contexts into one WhatsApp message + CTA + rationale.

Trigger kinds (why-now events)

Kind	Meaning
research_digest	New research/news relevant to the category
regulation_change	Rule/compliance change with a deadline
recall_due	Customer due for a revisit (e.g. 6-month cleaning)
perf_dip / perf_spike	Performance down / up
curious_ask_due	Light check-in question
festival_upcoming	Festival coming (e.g. Diwali)
competitor_opened	New competitor nearby on Maps
ipl_match_today	Cricket match affecting restaurants
chronic_refill_due	Pharmacy refill reminder
...	Many more in the dataset

API / engineering terms

Term	Meaning
API	Way for programs to talk over HTTP (URLs + JSON)
Endpoint	One URL path your bot exposes, e.g. /v1/healthz
HTTP	Web request protocol (GET read, POST send data)
JSON	Text data format { "key": "value" }
FastAPI	Python framework for building APIs quickly
Uvicorn	Server that runs the FastAPI app
Stateful	Bot remembers contexts/conversations between calls (in memory)
Idempotent	Same request twice doesn't break things (same context version = no-op)
Suppression key	Dedup id so the same nudge isn't spammed
Timeout	Max wait (judge waits 30 seconds per call)
Health check / healthz	"Are you alive?" endpoint
Metadata	Team name, model, approach info for the leaderboard
Dockerfile	Recipe to package the app into a container
Container / Docker	Isolated runnable box with your app + dependencies

Term	Meaning
Environment variable	Config like API keys (VERA_LLM_API_KEY) not hard-coded in files
<code>.env</code>	Local file holding secrets (gitignored — never upload)
Git	Version control for code history
GitHub	Website hosting Git repos
Commit	Snapshot of code saved in Git
Push	Upload commits to GitHub
Repo / repository	Project folder on GitHub
CLI	Command-line tool (gh, flyctl, curl)

Hosting / deploy terms

Term	Meaning
Localhost	Your own computer (<code>http://127.0.0.1:8080</code>) — Magicpin cannot reach this
Public URL	Internet address anyone (including the judge) can open
Deploy	Put the bot on a cloud computer so it runs without your laptop
Tunnel (Cloudflare)	Temporary bridge: internet → your Mac. Free but needs Mac on
<code>*.trycloudflare.com</code>	Temporary Cloudflare tunnel hostname
Fly.io	Cloud host; free tier now usually needs a credit card
Hugging Face Spaces	Host ML/apps; Docker spaces now often need PRO
Back4app Containers	Cloud Docker host from GitHub; free tier exists but temp URLs may expire
<code>*.b4a.run</code>	Back4app runtime URL
Permanent vs temporary URL	Permanent stays; temporary (e.g. 60 min) dies — bad for judging
Keep-alive	Script/process that pings the app / prevents Mac sleep

AI / LLM terms

Term	Meaning
LLM	Large Language Model (ChatGPT-like model)
Groq	Fast LLM API provider (free tier with API key)
API key	Secret password for an API (gsk_... for Groq)
Deterministic composer	Rule/template message builder that works without an LLM
LLM polish	Optional: rewrite the template message with Groq for nicer wording
Temperature = 0	Make LLM outputs as repeatable as possible
Hallucination	Model invents facts not in context — penalized heavily
Judge / judge harness	Magicpin's system that calls your bot and scores it
Judge simulator	Local script <code>judge_simulator.py</code> to practice scoring
Rubric	Scoring dimensions (each 0–10)

Scoring dimensions (0–10 each)

Dimension	Meaning
Decision quality	Did you pick the right signal / sometimes send nothing?
Specificity	Real numbers, offers, names from context
Category fit	Right tone for dentists vs salons vs restaurants
Merchant fit	Personalized to that shop's metrics/history
Engagement compulsion	Clear, easy reason to reply now

3. What Magicpin's judge calls (the contract)

Your bot must expose these endpoints:

Method	Path	Purpose
POST	<code>/v1/context</code>	Judge pushes category/merchant/customer/trigger data
POST	<code>/v1/tick</code>	Periodic wake-up; bot may return proactive messages
POST	<code>/v1/reply</code>	Merchant/customer replied; bot returns <code>send / wait / end</code>
GET	<code>/v1/healthz</code>	Liveness ("I'm up")

Method	Path	Purpose
GET	/v1/metadata	Team + model info

We also added:

Method	Path	Purpose
GET	/	Friendly “bot is online” page (not required by judge)
POST	/v1/teardown	Optional wipe of in-memory state

Important judge rules

- Respond within **~30 seconds**
- Max **20 actions** per /v1/tick
- Empty {"actions": []} is OK (restraint is good)
- Don't invent offers/competitors/numbers
- Detect auto-replies and hostile “stop”
- On “ok let's do it”, switch to **action** (don't keep asking qualifying questions)

4. Project file map

```
vera-ai-challenge/
├─ bot.py           # FastAPI server – all HTTP endpoints
├─ composer.py      # Builds WhatsApp messages from 4 contexts
├─ handlers.py      # Multi-turn reply logic (auto-reply / intent /
hostile)
├─ state.py         # In-memory storage for contexts & conversations
├─ requirements.txt  # Python packages
├─ Dockerfile       # How to run in Docker/cloud
├─ .env             # Secrets locally (NOT on GitHub)
├─ .env.example     # Safe template for env vars
├─ .gitignore       # What Git must never upload
├─ README.md        # Short project readme
├─ DEPLOY.md        # Cloud deploy instructions
├─ PROJECT-GUIDE.md # This document
├─ deploy.sh        # Fly.io deploy script (needs card)
├─ deploy-hf.sh     # Hugging Face deploy (needs PRO for Docker)
├─ keep-alive.sh    # Keep Mac + Cloudflare tunnel awake
├─ fly.toml         # Fly.io config
├─ judge_simulator.py # Official local judge practice tool
├─ challenge-brief.md # Official product/challenge brief
├─ challenge-testing-brief.md # Official API/test contract
├─ engagement-design.md # Messaging design notes
```

```
|— engagement-research.md # Research notes
|— dataset/               # Seeds + generator
|   |— categories/       # dentists, salons, ...
|   |— *_seed.json
|   |— generate_dataset.py
|— examples/             # API examples + case studies
|— expanded/             # Generated full dataset (local; gitignored)
```

Main code roles

1. **bot.py** — Receives HTTP calls, stores context, calls composer/handlers
 2. **composer.py** — `compose(category, merchant, trigger, customer?)` → message
 3. **handlers.py** — Interprets merchant replies
 4. **state.py** — Memory during one running process
-

5. Step-by-step — everything we did so far

Step 1 — Understood the assignment

- Opened <https://magicpin.com/vera/ai-challenge>
- Learned: build Vera's message engine from 4 contexts; expose 5 HTTP endpoints; submit a **public URL**

Step 2 — Created the project

- Folder: `~/Projects/vera-ai-challenge`
- Downloaded official zip from Magicpin partners site
- Unzipped briefs, dataset seeds, `judge_simulator.py`

Step 3 — Generated the full dataset

```
python3 dataset/generate_dataset.py --seed-dir dataset --out expanded
```

Produces ~50 merchants, ~200 customers, ~100 triggers, 30 test pairs.

Step 4 — Built the bot

Implemented: - All required endpoints - Trigger-kind message templates (grounded in real context fields) - Reply handling: - auto-reply → end - affirmative ("ok let's do it") → action mode - hostile/stop → end - Optional Groq LLM polish if `VERA_LLM_API_KEY` is set

Step 5 — Ran locally

```
uvicorn bot:app --host 0.0.0.0 --port 8080
```

Smoke-tested healthz, tick, reply scenarios successfully.

Step 6 — Set team identity

```
VERA_TEAM_NAME=Lovepreet  
VERA_TEAM_MEMBERS=Lovepreet Singh  
VERA_CONTACT_EMAIL=lovepreetsingh40888@gmail.com
```

Step 7 — Added Groq LLM

- Created Groq API key
- Stored in `.env` (not committed)
- Metadata shows model `llama-3.3-70b-versatile`

Step 8 — Temporary public URL via Cloudflare tunnel

- Installed `cloudflared`
- Exposed localhost to a `*.trycloudflare.com` URL
- **Limitation:** Mac must stay on; URL can change if tunnel restarts
- User rejected long-term Mac dependency

Step 9 — Tried Fly.io (cloud, no Mac)

- Installed `flyctl`, prepared `Dockerfile` + `fly.toml` + `deploy.sh`
- Login worked
- **Blocked:** Fly required payment/card info to create the app

Step 10 — Tried Hugging Face Spaces

- Script `deploy-hf.sh`, login as `love2902` succeeded
- **Blocked:** Docker Spaces require Hugging Face **PRO** (402 Payment Required)

Step 11 — GitHub backup + Back4app path

- Fixed GitHub CLI auth (`lovepreet166`)
- First commit created (`.env` excluded)
- Private repo: <https://github.com/lovepreet166/vera-ai-challenge>
- Documented Back4app free container deploy in `DEPLOY.md`

Step 12 — Deployed on Back4app

- Created Web Deployment from GitHub repo
- App came online at:
- <https://veraaichallenge-r5txgnlv.b4a.run/>
- Verified:
- / → online JSON
- /v1/healthz → ok
- /v1/metadata → team + Groq model
- /v1/tick → ok
- **Caveat:** Back4app showed **temporary URL (~60 minutes)** unless upgraded to permanent

Step 13 — Submission constraint clarified

Magicpin needs the bot **active throughout judging**.

A 60-minute temporary URL is **not safe** to submit unless upgraded to a **permanent** URL (or another always-on host).

6. How the bot decides messages (simple flow)

```

Judge POST /v1/context → store category/merchant/trigger/customer
      ↓
Judge POST /v1/tick    → for each active trigger:
                        skip if expired/suppressed
                        compose(category, merchant, trigger, customer?)
                        return actions[] (or [])
      ↓
Judge plays merchant reply
      ↓
Judge POST /v1/reply   → auto-reply / yes / stop / normal
                        → send | wait | end

```

Example of a strong message (from case studies)

Weak: “Hi Doctor, want a discount campaign?”

Strong: “190 people in your locality are searching for Dental Check Up. Should I send Cleaning @ ₹299?”

7. How to run locally (for you)

```

cd ~/Projects/vera-ai-challenge
source .venv/bin/activate

```

```
# .env already has team + Groq key
uvicorn bot:app --host 0.0.0.0 --port 8080
```

Open: - <http://127.0.0.1:8080/> - <http://127.0.0.1:8080/docs> (interactive API docs) -
<http://127.0.0.1:8080/v1/healthz>

Optional local judge practice (needs an LLM key inside `judge_simulator.py` for scoring):

```
export BOT_URL=http://localhost:8080
python3 judge_simulator.py
```

8. Submission checklist

Requirement	Status
Bot implements 5 endpoints	Done
Team name/email in metadata	Done
Groq LLM optional polish	Done
Code on GitHub	Done
Public cloud URL works	Done (Back4app)
URL permanent for whole judging window	Must confirm / upgrade
Mac can be off	Only if URL is permanent cloud

Safe submit URL rule

Submit a base URL like:

```
https://veraaichallenge-r5txgnlv.b4a.run
```

Only if the dashboard says it will **stay active** (not “temporary 60 minutes”).

9. Secrets & safety

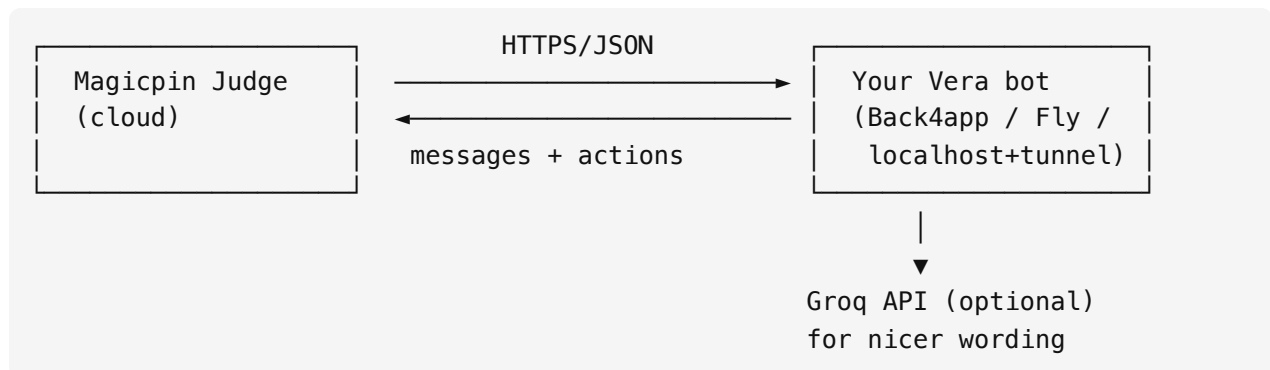
Item	Where	Never do
Groq API key	.env + cloud env vars	Don't commit to GitHub / don't paste in public chat long-term
Fly/HF tokens	Local CLI login	Don't share
Merchant dataset	Synthetic challenge data	Fine to use in this project

If a key was pasted in chat, **rotate it** later at the provider dashboard.

10. Current recommended next actions

1. On Back4app, check if URL is temporary or permanent.
2. If temporary → **Upgrade for Permanent URL** (or move to Fly/AWS with a card).
3. Re-test /v1/healthz on the final URL.
4. Submit that **permanent** base URL on <https://magicpin.com/vera/ai-challenge>
5. Keep the cloud app running until results; Mac can sleep only if the cloud URL is permanent.

11. Quick “who talks to whom” diagram



12. One-sentence summary

We built a WhatsApp-style merchant AI bot for Magicpin’s Vera challenge, packaged it as a FastAPI service with real context-grounded messaging and reply handling, pushed it to GitHub, and deployed it to Back4app — submission is ready once the public URL is confirmed permanent for the full judging period.

Last updated: 19 Sep 2026

For deploy clicks see `DEPLOY.md`. For short overview see `README.md`.